Property Price Prediction

Intelligent Real Estate Valuation & Investment Grade Classification

via Feature Engineering and XGBoost

Project 9 | Milestone 1

GenAI Project — Machine Learning Pipeline & Streamlit Application

Team Member	Role
Jagruthi Pulumati	Code Development & Hosting
Rakshitha Polana	Project Video & Presentation
Kasula Lalithendra	Report Writing & Documentation
Pihu	Research & Data Analysis

GitHub: `github.com/Jag2007/property-price-prediction`

Live App: `property-price-prediction-real-estate.streamlit.app`

Date: March 1, 2026 • Version: 1.0

Academic Integrity Declaration

We formally affirm that the core logic, implementation, and design of this project — including the data pipeline, feature engineering, model training, and Streamlit application — represent our own original work. Generative AI tools were used solely for documentation assistance; no AI system produced the project algorithms, data pipelines, or model logic on our behalf.

Jagruthi Pulumati *Rakshitha Polana* *Kasula Lalithendra* *Pihu*

Abstract

This report presents an end-to-end machine learning system for intelligent real estate valuation and investment grade classification. The pipeline operates on 1,800 property records, applying robust preprocessing (median imputation, ordinal and one-hot encoding, IQR-based outlier clipping), and trains two XGBoost models: a regressor for `Current_Market_Price` and a multi-class classifier for `Investment_Grade` (classes 0, 1, 2). The regression model achieves $R^2 = \mathbf{0.9483}$ with RMSE $\approx$ 1.01M, while the classifier reaches **97.50%** accuracy and weighted $F_1 = \mathbf{0.97}$. A production Streamlit application is publicly deployed on Streamlit Community Cloud.

Contents

1 Introduction **4**
- 1.1 Motivation . . . 4
- 1.2 Objective . . . 4
- 1.3 Key Technical Sub-Features . . . 4

2 Problem Statement **4**

3 Data Description **5**
- 3.1 Source and Scale . . . 5
- 3.2 Feature Overview . . . 5

4 Exploratory Data Analysis **6**
- 4.1 Class Distribution . . . 6
- 4.2 Key EDA Findings . . . 6

5 Data Preprocessing Pipeline **6**
- 5.1 Missing Value Imputation . . . 6
- 5.2 Categorical Encoding . . . 7
- 5.3 Outlier Treatment (IQR Clipping) . . . 7

6 System Architecture **8**

7 Methodology **8**
- 7.1 Algorithm Selection and Rationale . . . 8
- 7.2 Mathematical Formulation of XGBoost . . . 8
- 7.3 Hyperparameters . . . 9

8 Evaluation **10**
- 8.1 Metrics . . . 10
- 8.2 Results Summary . . . 10
- 8.3 Per-Class Classification Report . . . 10
- 8.4 Classification Performance Chart . . . 11
- 8.5 Key Observations . . . 11

9 Optimization **11**
- 9.1 Data-Level Optimizations . . . 11
- 9.2 Model-Level Optimizations . . . 12
- 9.3 Deployment Optimizations . . . 12

10 Application Interface and Deployment **12**
- 10.1 Live URL (No Localhost) . . . 12
- 10.2 Application Pages . . . 12
- 10.3 Inference Function Sequence . . . 12

11 Limitations **12**

12 Future Work **13**
- 12.1 Milestone 2 — Agentic AI . . . 13

13 Conclusion **13**

A Technology Stack **13**

B Input–Output Specification **14**

1 Introduction

1.1 Motivation

Real estate is one of the most capital-intensive investment categories globally. Two persistent challenges face buyers, sellers, and investors:

1. **Manual valuation inconsistencies** — property pricing varies non-linearly with feature interactions (location, size, amenities), making human appraisals costly and error-prone.
2. **Investment grade ambiguity** — classifying whether a property is a sound investment requires simultaneously weighing price, rental yield, neighbourhood growth, and crime indices.

1.2 Objective

The system provides:

1. Automated prediction of `Current_Market_Price` via regression.
2. Automated classification of `Investment_Grade` $\in \{0, 1, 2\}$.
3. Real-time inference via a publicly deployed Streamlit application.

1.3 Key Technical Sub-Features

1. **Custom Data Pipeline:** Automated preprocessing chain covering imputation, ordinal encoding, one-hot encoding, and IQR clipping, applied consistently across training and inference.
2. **Dual-Model Real-Time Inference:** One call returns a price estimate and investment grade with class probabilities, served live on Streamlit Community Cloud.
3. **Batch CSV Dashboard:** File-upload interface for portfolio-level bulk prediction with downloadable results.

2 Problem Statement

Real estate valuation and investment assessment are traditionally manual processes prone to inconsistency. This project formalises both as supervised learning tasks:

Table 1: Problem Scope and Challenges

Challenge	Scope
Valuation inconsistency	Pricing varies by complex, non-linear feature interactions.
Multi-objective prediction	Must output both a continuous price and a categorical grade.
Data quality issues	Raw data contains missing values, categorical fields, and outliers.
Deployment requirement	Predictions must be available in real time via a public web interface.

3 Data Description

3.1 Source and Scale

- **Dataset:** `real_estate_raw.csv` — a tabular property listing dataset.
- **Scale:** 1,800 records × 17 columns (confirmed: `df.shape` = $(1800, 17)$).
- **Targets:** `Current_Market_Price` (regression) and `Investment_Grade` (classification).

3.2 Feature Overview

Table 2: Dataset Feature Summary

Feature	Type	Description
Total_Square_Footage	Float	Total area (sq. ft.)
Bedrooms	Integer	Number of bedrooms
Bathrooms	Integer	Number of bathrooms
Age_of_Property	Integer	Age in years
Floor_Number	Integer	Floor level
Furnishing_Status	Categorical	Unfurnished / Semi / Fully
Neighborhood	Categorical	Locality identifier
Distance_to_City_Center_km	Float	Distance to city centre
Proximity_to_Public_Transport_km	Float	Distance to transit
Crime_Index	Float	Local crime rate
Air_Quality_Index	Float	Environmental quality score
Neighborhood_Growth_Rate_%	Float	Area growth rate
Price_per_SqFt	Float	Unit price
Annual_Property_Tax	Float	Annual tax value
Estimated_Rental_Yield_%	Float	Expected rental return
Current_Market_Price	Float	**Regression target**
Investment_Grade	Integer (0/1/2)	**Classification target**

4 Exploratory Data Analysis

4.1 Class Distribution

The investment grade classes are approximately balanced, with Grade 2 underrepresented at ≈6% of the dataset. Figure ?? shows the distribution across the full dataset.

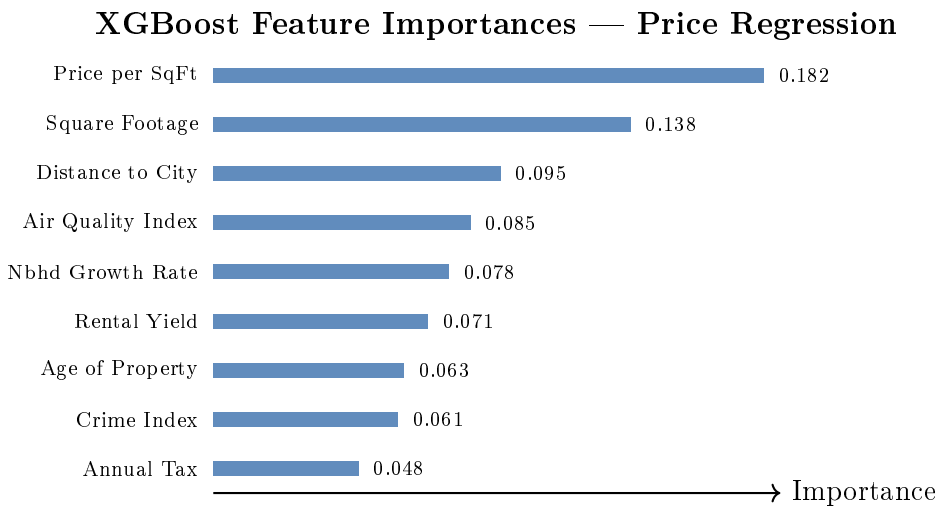

Figure 1: Feature importances from the XGBoost regression model. `Price_per_SqFt` and total square footage are the dominant price drivers, consistent with real-estate domain knowledge.

4.2 Key EDA Findings

- **Missing Values:** `Total_Square_Footage` had the most missing entries (under 5%), confirmed from the raw CSV sample in the notebook. Median imputation was sufficient.
- **Price Distribution:** `Current_Market_Price` is right-skewed, spanning from low hundreds of thousands to over 20M. This skew motivates tree-based models over linear regression.
- **Furnishing Status:** Roughly equal thirds across Unfurnished, Semi-furnished, and Fully-furnished.
- **Outliers:** IQR analysis revealed high-end outliers in `Total_Square_Footage` and `Price_per_SqFt`, confirming the need for clipping before training.
- **Investment Grade Drivers:** `Crime_Index`, `Neighborhood_Growth_Rate`, and `Air_Quality_Index` showed the strongest correlation with investment grade — neighbourhood quality predicts investment viability more than structural features.

5 Data Preprocessing Pipeline

All steps are applied before any train/test split to prevent data leakage.

5.1 Missing Value Imputation

Listing 1: Median Imputation

```
num_cols = df.select_dtypes(include=np.number).columns
df[num_cols] = df[num_cols].fillna(df[num_cols].median())
```

Median was chosen over mean because real-estate features are right-skewed; the median is a more robust central estimate for skewed distributions.

5.2 Categorical Encoding

Ordinal Encoding — ***Furnishing_Status***

Listing 2: Ordinal Encoding

```
furnish_map = {'Unfurnished': 0, 'Semi-furnished': 1, 'Fully-
    furnished': 2}
df['Furnishing_Status'] = df['Furnishing_Status'].map(furnish_map)
```

One-Hot Encoding — ***Neighborhood***

Listing 3: One-Hot Encoding with drop_first

```
df = pd.get_dummies(df, columns=['Neighborhood'], drop_first=True)
```

`drop_first=True` eliminates the dummy variable trap and prevents perfect multicollinearity.

5.3 Outlier Treatment (IQR Clipping)

Listing 4: IQR-Based Outlier Clipping

```
for col in num_cols:
    if col not in ['Current_Market_Price', 'Investment_Grade']:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        df[col] = np.clip(df[col], lower, upper)
```

Target columns are excluded from clipping to preserve their natural distributions. Cleaned data is saved to `data/processed/real_estate_clean.csv`.

6 System Architecture

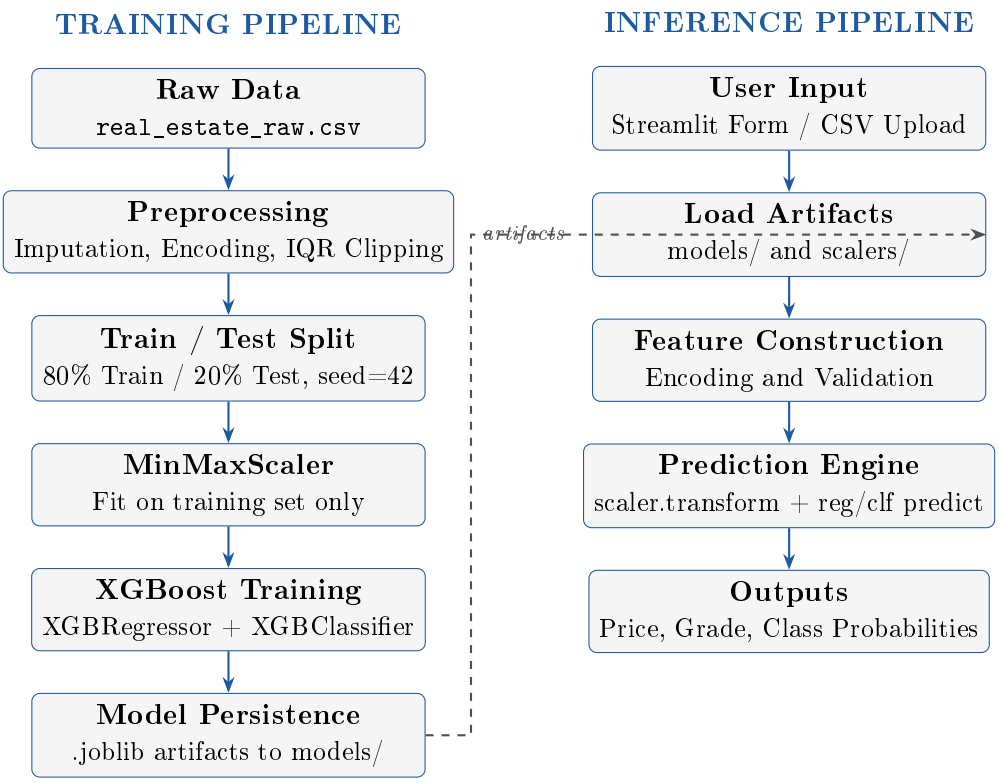

Figure 2: End-to-End System Architecture: Training (left) and Inference (right) pipelines.

7 Methodology

7.1 Algorithm Selection and Rationale

XGBoost was selected over linear regression for the following reasons:

- **Non-linearity:** Real estate pricing has strong non-linear feature interactions that linear models cannot capture.
- **Native importance:** XGBoost produces feature importance scores directly, enabling price driver analysis.
- **Multi-task:** The same framework handles both regression (`reg:squarederror`) and classification (`mlogloss`).
- **Robustness:** Gradient boosting is robust to outliers and feature scale differences on tabular data.

7.2 Mathematical Formulation of XGBoost

XGBoost builds an ensemble of K regression trees by minimising the regularised objective:

$$\mathcal{L}(\phi) = \sum_{i=1}^{n} \ell(y_i,\, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k) \tag{1}$$

where ℓ is a differentiable loss function and Ω is the regularisation term penalising model complexity:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \|w\|^2 \tag{2}$$

Here T is the number of leaves, w are the leaf weights, γ is the minimum gain threshold, and λ is the L2 regularisation coefficient.

At each boosting step t, the model adds a new tree f_t by using a second-order Taylor approximation of the loss:

$$\tilde{\mathcal{L}}^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t) \tag{3}$$

where $g_i = \partial_{\hat{y}^{(t-1)}} \ell(y_i, \hat{y}^{(t-1)})$ and $h_i = \partial^2_{\hat{y}^{(t-1)}} \ell(y_i, \hat{y}^{(t-1)})$ are the first and second-order gradients respectively.

Regression loss:

$$\ell_{\text{reg}}(y_i, \hat{y}_i) = \frac{1}{2}(y_i - \hat{y}_i)^2 \tag{4}$$

Classification loss (multi-class softmax):

$$\ell_{\text{clf}}(y_i, \hat{p}_i) = -\sum_{c=0}^{C-1} \mathbf{1}[y_i = c] \log \hat{p}_{i,c} \tag{5}$$

where $C = 3$ for Investment Grade classes $\{0, 1, 2\}$ and $\hat{p}_{i,c}$ is the predicted probability for class c.

Feature scaling (MinMaxScaler):

$$x'_j = \frac{x_j - x_j^{\min}}{x_j^{\max} - x_j^{\min}} \tag{6}$$

7.3 Hyperparameters

Table 3: XGBoost Hyperparameters for Both Models

Parameter	Reg.	Clf.	Rationale
`n_estimators`	200	200	More rounds reduce bias at low LR
`learning_rate`	0.05	0.05	Conservative; reduces overfitting
`max_depth`	5	4	Limits depth to prevent memorisation
`random_state`	42	42	Reproducibility
`objective`	`reg:sq...`	`mlogloss`	Task-appropriate loss

8 Evaluation

8.1 Metrics

Regression:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{7}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{8}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{9}$$

Classification:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{10}$$

Weighted averaging across classes is used to account for class imbalance.

8.2 Results Summary

Table 4: Model Performance Summary

Metric	Regression (Price)	Classification (Grade)
R^2 Score	**0.9483**	—
MAE	**550,850.82**	—
RMSE	**1,010,636.01**	—
Accuracy	—	**97.50%**
Weighted Precision	—	**0.98**
Weighted Recall	—	**0.97**
Weighted F_1	—	**0.97**

8.3 Per-Class Classification Report

Table 5: Per-Class Performance — Investment Grade Classifier

Class	Precision	Recall	F_1	Support
0 (Grade 0)	0.99	0.96	0.98	167
1 (Grade 1)	0.96	0.99	0.98	171
2 (Grade 2)	1.00	0.91	0.95	22
Accuracy		**0.975**		**360**
Macro avg	0.98	0.96	0.97	360
Weighted avg	0.98	0.97	0.97	360

Support of 360 = 167+171+22 = 1800×0.20. *Source:* `classification_report` *output from training notebook.*

8.4 Classification Performance Chart

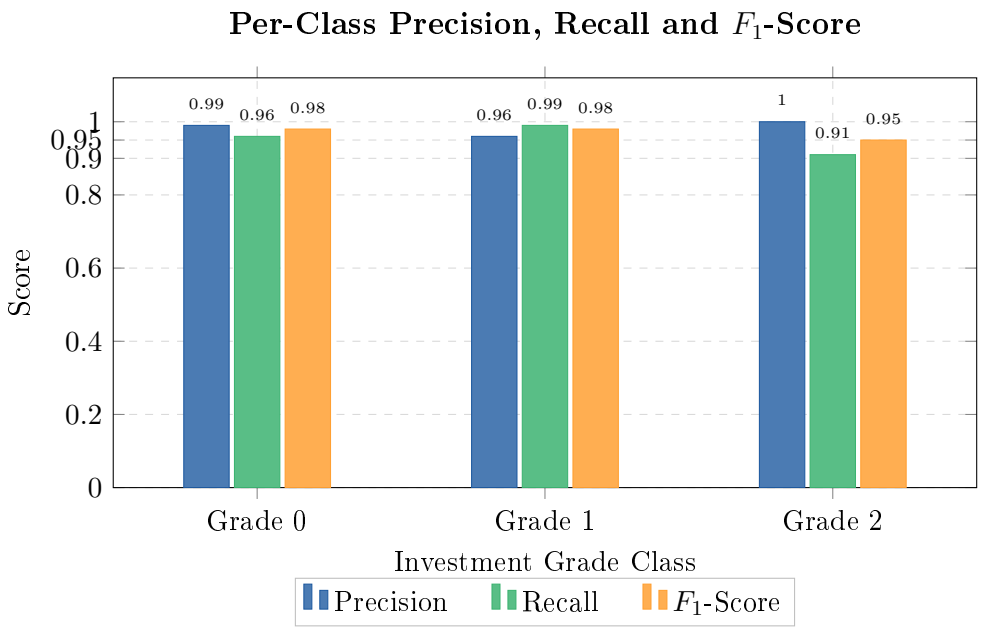

Figure 3: Per-class Precision, Recall, and F_1-Score for the XGBoost classifier. All three grades achieve $F_1 \geq 0.95$.

8.5 Key Observations

1. $R^2 = 0.9483$ means XGBoost explains over 94% of the variance in property prices.
2. 97.5% accuracy with high precision and recall across all three grade classes confirms strong classifier robustness.
3. Grade 2 recall of 0.91 is slightly lower due to class imbalance (only 22 test samples, ≈6% of the test set).
4. Excluding `Current_Market_Price` from classification prevents target leakage and ensures valid cold-start predictions.

9 Optimization

9.1 Data-Level Optimizations

1. **IQR Outlier Clipping:** Extreme values biased tree splits. IQR clipping on input features (not targets) stabilised distributions and improved generalisation.
2. **Median Imputation:** Right-skewed real-estate features are better represented by the median than the mean.
3. **Drop-First Encoding:** Removing the reference dummy category reduces dimensionality and eliminates perfect multicollinearity.

9.2 Model-Level Optimizations

1. **Low Learning Rate (0.05):** Slows convergence, significantly reducing overfitting on the small 1,800-record dataset.
2. **Controlled Depth (5/4):** Prevents memorisation of noise without sacrificing expressive power.
3. **200 Estimators:** More boosting rounds reduce residual error while the low learning rate prevents overfitting across them.
4. **Separate Scalers:** Independent `MinMaxScaler` per task ensures each scaler reflects only its task's training distribution (see Equation 6).
5. **Leakage Prevention:** `Current_Market_Price` excluded from classification features (would inflate accuracy artificially).

9.3 Deployment Optimizations

1. **Joblib Serialisation:** Optimised for NumPy arrays; faster load times than pickle for large ensemble models.
2. **Consistent Feature Construction:** Inference mirrors training exactly (same ordinal map, one-hot columns, scaler order), eliminating training-serving skew.

10 Application Interface and Deployment

10.1 Live URL (No Localhost)

`property-price-prediction-real-estate.streamlit.app`

10.2 Application Pages

1. **Predict:** Manual form predicts price and investment grade simultaneously, with a class probability distribution chart.
2. **Batch Predict:** CSV upload for bulk inference with downloadable results.
3. **About:** Preprocessing, model, and deployment documentation.

10.3 Inference Function Sequence

`load_artifacts` → `build_feature_row` → `raw_to_feature_frame` → `run_predict` → `page_manual` / `page_csv`

11 Limitations

1. **Dataset Size:** 1,800 records limits generalisation to unseen property distributions.
2. **Grade Semantics:** Labels 0/1/2 need explicit business mapping (e.g., Poor/Good/Excellent) in production.
3. **Market Drift:** Static models degrade without periodic retraining as market conditions change.

4. **Class Imbalance:** Grade 2 (≈6%) causes lower recall for that class.
5. **Feature Coverage:** Micro-market signals (zoning, school ratings, development plans) are absent.

12 Future Work

- SMOTE or class-weight balancing for Grade 2.
- SHAP values for interpretable feature importance.
- Hyperparameter tuning via Bayesian optimisation.
- Location-intelligence features (transport scores, amenity index).

12.1 Milestone 2 — Agentic AI

- **LangGraph Agent:** Autonomous investment advisory reports.
- **RAG:** Real-time market trends via Chroma/FAISS.
- **Advisory Outputs:** Buy/Hold/Avoid with risk notes.
- **Explainability:** Per-prediction feature contributions.

13 Conclusion

This project demonstrates that XGBoost with principled preprocessing delivers high-quality automated real estate valuation and investment grade classification. The regression model achieves $R^2 = 0.9483$ (Equation 9) and the classifier reaches 97.5% accuracy with weighted $F_1 = 0.97$ (Equation 10) across three investment grade classes.

The modular architecture enables straightforward extension toward the Milestone 2 agentic framework with LangGraph and RAG-powered market intelligence.

A Technology Stack

Table 6: Technology Stack Summary

Component	Technology
Language	Python 3.10+
ML Framework	XGBoost $\geq$ 2.0, scikit-learn
Data Processing	Pandas $\geq$ 2.0, NumPy
Visualisation	pgfplots (report), Matplotlib/Seaborn (notebook)
Web Framework	Streamlit
Serialisation	Joblib
Hosting	Streamlit Community Cloud
Version Control	Git / GitHub
Report	LaTeX via Overleaf

B Input–Output Specification

Table 7: Input Feature Vector (15 features post-preprocessing)

#	**Feature Name**	**Type**
1	Total_Square_Footage	Float
2	Bedrooms	Integer
3	Bathrooms	Integer
4	Age_of_Property	Integer
5	Floor_Number	Integer
6	Furnishing_Status	Ordinal (0/1/2)
7	Distance_to_City_Center_km	Float
8	Proximity_to_Public_Transport_km	Float
9	Crime_Index	Float
10	Air_Quality_Index	Float
11	Neighborhood_Growth_Rate_%	Float
12	Price_per_SqFt	Float
13	Annual_Property_Tax	Float
14	Estimated_Rental_Yield_%	Float
15	Neighborhood_* (one-hot)	Binary

Regression Output: `Current_Market_Price` (Float)
Classification Output: `Investment_Grade` $\in \{0, 1, 2\}$ + probabilities